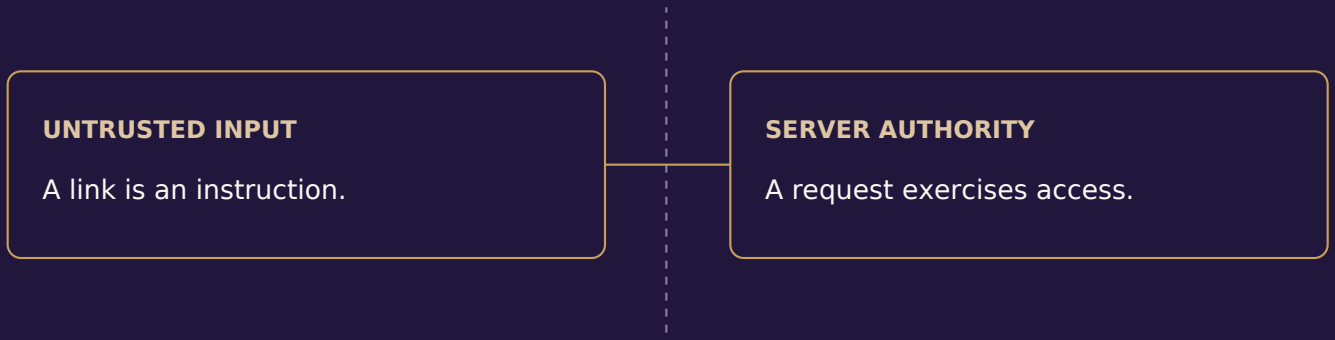
SERVER TRUST / NETWORK AUTHORITY / SECURE DESIGN

SSRF Explained

Understand server trust, find and prove request forgery, and build safer outbound systems



Understand the route before the vulnerability.
Find the failure. Prove the effect. Repair the boundary.
Practice with dummy services on your own computer.

Haroon Shahid

First edition / Version 1.0 / 26 October 2026

Before you begin

SSRF Explained

Understand server trust, find and prove request forgery, and build safer outbound systems

By Haroon Shahid. First edition, version 1.0. 26 October 2026.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under LICENSE.txt. The separately identified lab code is MIT licensed. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and destinations you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Framework, cloud, parser, and protocol guidance is version-sensitive. The local lab uses real loopback HTTP, injected resolver changes, and fictional metadata. It is not a production security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package includes all 56 chapters, answered reviews, the offline reading edition, practice workbooks, reference sheets, and the local Request Foundry lab.

Contents

Before you begin 2

Contents 3

Introduction 4

Learning roadmap 5

01. One click, two requests 8

02. Addresses, ports, and routes 10

03. DNS is a lookup, not permission 12

Sources and further reading 14

Introduction

A customer pastes a link. A server retrieves it. Something useful happens: a preview appears, an image is imported, or an integration is tested. The same useful behavior can become a security failure when the input borrows network access or identity it should not control.

This book teaches that boundary from the beginning. You do not need previous networking or security knowledge. We first separate the browser request from the server request. Then we build the ideas of addresses, DNS, URLs, HTTP, identity, and trust. Only then do we begin finding and proving request forgery.

The running story is Parcel Press, a fictional publishing service. Its previews, imports, webhooks, and workers make the ideas concrete. The companion lab, Request Foundry, gives you a closed track for testing. You can watch the actual local peer, follow a redirect, see a dummy profile, and verify that a repair prevents contact.

The central habit is simple: **predict, run, inspect, explain, repair.** Before an exercise, write what you expect. After it, compare the input, decision, connection, and result. If your prediction was wrong, find the missing assumption. That is how a security mental model grows.

You will learn exploitation through bounded demonstrations, not through attacks on strangers' systems. Our examples use dummy data, reversible actions, fixed local destinations, and explicit permission boundaries. The purpose is to make you capable of identifying and repairing real failures in systems you are authorized to assess.

Depth includes knowing why a proposed attack does not work. A GET-only sink may not complete a token-gated sequence. A callback may prove contact without readback. A private service may require independent authentication. Understanding these limits makes your conclusions stronger.

This is a substantial learning resource, not a certification or a promise that one book makes anyone an expert. Independent skill comes from careful practice, implementation, peer review, and keeping current. The final chapters show how to produce work that another person can verify.

How to use the package

Read the PDF for fixed pages and a linked topic index. Use the offline HTML for responsive text, scalable diagrams, and answer controls. Both contain the complete book. Work through the prediction deck, local missions, review clinic, and mastery tracker. Use the source catalog to investigate a rule beyond the explanation here.

The source modules and original diagrams are included in the separate publisher materials. The paid reader package has a clear start page and does not require any seller setup. Runtime requirements and test scope are in its lab guide and validation report.

Three habits to keep

First, distinguish input acceptance from actual contact. Second, distinguish contact from consequence. Third, check that a repair prevents the forbidden operation rather than merely hiding its response. These habits recur because they solve different evidence problems, not because you need to memorize a slogan.

Learning roadmap

Choose your starting route

Reader	Begin here	Produce this evidence
New to web security	Chapters 1-8, then 9-16	A two-leg request map and one bounded lab proof
Developer with HTTP experience	Read 1, 4, 7, 8; then 17-24 and 41-48	A connector contract and regression matrix
Authorized security tester	Read 8-16, then 17-40	A capability matrix and a precise finding
Architect or reviewer	Read 8, 25-48, then 49-56	A consumer inventory and target architecture

Skipping a route is optional. If a later explanation feels unclear, return to the glossary's introducing chapter. You do not need to struggle through an unknown term.

Seven parts, one connected model

1. Chapters 1-8: understand requests, addresses, names, URLs, identity, and authority.
2. Chapters 9-16: discover request makers and prove bounded failures.
3. Chapters 17-24: reason through parsing, resolution, connection, redirects, proxies, and identity.
4. Chapters 25-32: follow hidden fetches in documents, jobs, protocols, and agent tools.
5. Chapters 33-40: assess cloud, compound, content, and availability consequences.
6. Chapters 41-48: build narrower interfaces, clients, egress boundaries, and observability.
7. Chapters 49-56: review, repair, report, and maintain independent practice.

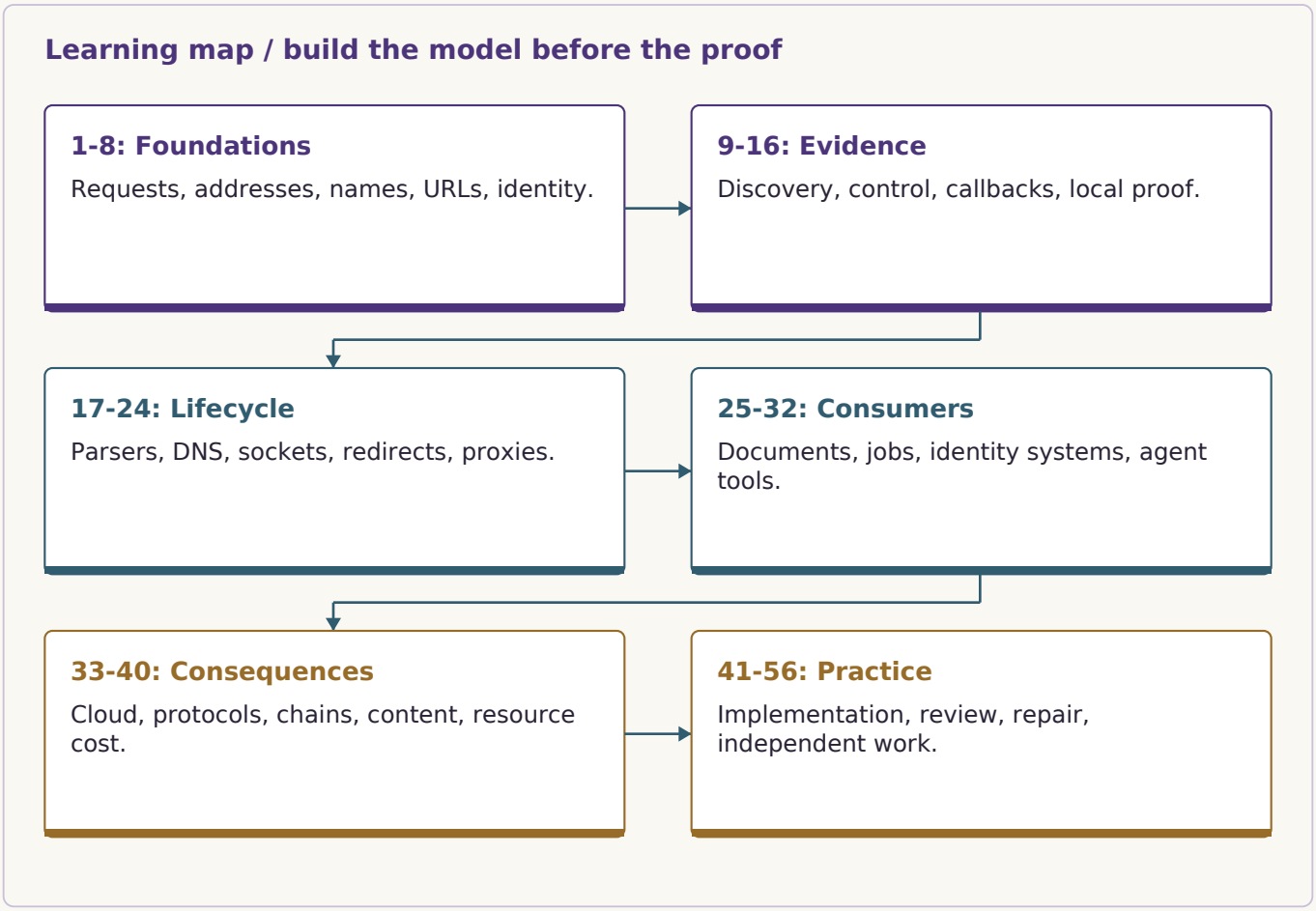
Pass gates instead of counting pages

Gate	You can explain	You can demonstrate
Foundations	Browser versus server authority	Label both request legs and credentials
Prediction	How a URL becomes a peer	Predict a redirect and resolver-switch trace
Bounded proof	Contact versus readback or action	Produce a dummy proof and negative control
Implementation	Approval preserved to the socket	Locate the connector invariant in code
Architecture	Multiple consumers and identities	Design independent app and egress controls
Independent review	Evidence, repair, and limits	Deliver the capstone review package

A flexible study rhythm

Use short sessions. Read one or two chapters, close the book, and explain the main idea aloud. Answer the checks before opening the explanations. In the next session, revisit one previous prediction. After several days, retry a worked case without its answer. Adjust the pace to your background; the package does not assume a fixed completion time.

Keep an error notebook: prediction, observation, missing assumption, corrected model, next test. Review that notebook before adding more techniques. It records how you think, not only what you have seen.



Read left to right within each stage, then follow the connector to the next row. Each later stage uses the earlier mental model.

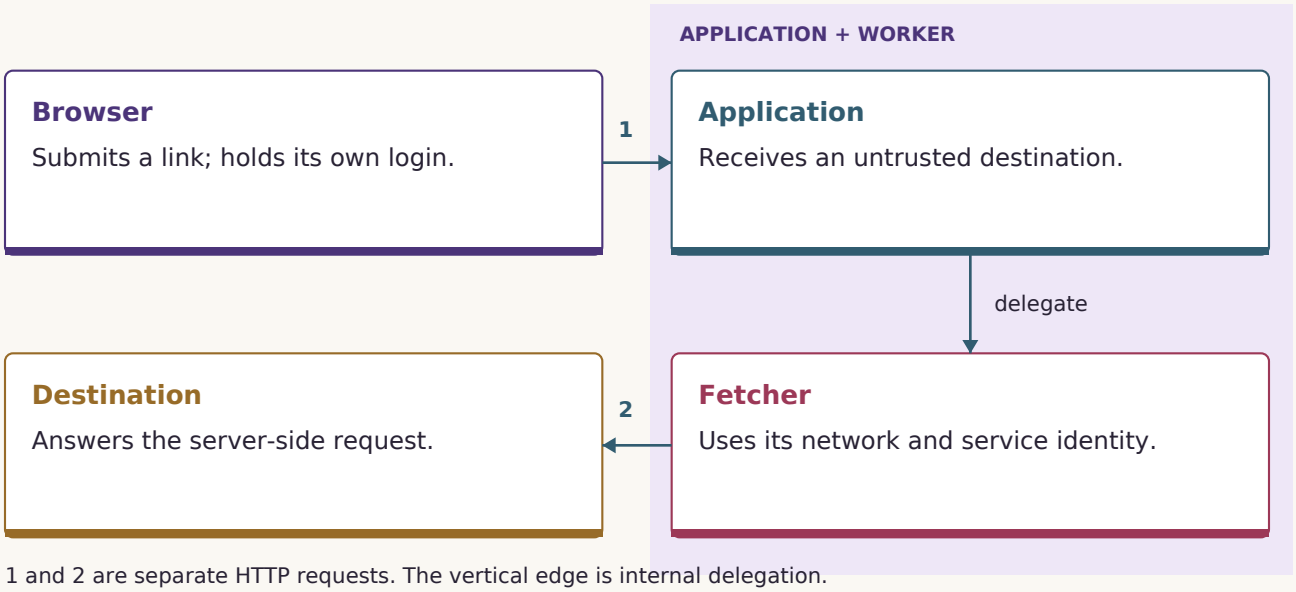
I. SEE THE NETWORK CLEARLY

01. One click, two requests

Learn to: Separate a browser request from a server request.

Think of it this way: A hotel guest asks the concierge to collect a parcel. The concierge uses a staff entrance the guest cannot use.

01A / Two requests, different authority



1 and 2 are separate HTTP requests. The vertical edge is internal delegation.

The application crosses from accepting data to exercising outbound authority. The browser and server have separate credentials and network positions.

Begin with a useful feature

Nora works on a small publishing service called Parcel Press. A customer pastes a link. The service fetches the page and creates a preview. This is normal product behavior. Many applications must contact other systems to do their work.

A **client** asks for something. A **server** answers. These are roles, not permanent kinds of computer. Your browser is a client of Parcel Press. Parcel Press then becomes a client of the website in the pasted link. There are two separate requests, with two separate sources of authority.

A **cookie** is a small value a browser can store and attach to matching requests. A **session identifier** can link a request to stored login state. A server-side client has its own rules and state; it does not automatically inherit the browser’s cookies.

Imagine you cannot enter a staff room, but a helpful employee can. If you can choose where the employee goes, you might make the employee cross a boundary for you. This is the central idea behind server-side request forgery, or SSRF. We will build the network model before examining the failure.

Follow the two legs

The first leg is browser to application. It may contain a login cookie and a JSON field named `url`. The second leg is application to destination. It has its own method, headers, network route, and response. The application may run on a private network. It may have service credentials. It may see a different DNS answer from your laptop.

Do not merge these legs in your notes. A cookie on the first request does not automatically become a cookie on the second. The second connection may come from a worker long after the first request ends. Conversely, an application can deliberately copy a header across the boundary. You must inspect the actual implementation.

Question	First leg	Second leg
Who opens the connection?	Your browser	The application or its delegate
Who chooses the destination?	Your navigation or client code	Application logic influenced by input
Where is authority checked?	Incoming endpoint	Outbound policy and destination

Make a prediction

You paste `https://news.example/article`. The preview appears. This proves the product can obtain content somehow. It does not yet prove an SSRF vulnerability. The feature may accept only approved providers, use an isolated fetch service, or serve an existing cache entry. A vulnerability requires an unauthorized boundary crossing, not simply an outbound request.

Our running question is therefore precise: **Can untrusted input make a component use its network access or identity beyond its intended permission?** The input may be a URL, a file, a message, or an identifier that becomes a URL later.

Practice before moving on

Draw two arrows for a link preview: browser to application, application to destination. Label the owner of each credential. Put a box around the application network. Now add a background worker. Which component actually opens the second connection? Keep that question beside every later example. It prevents a large class of mistaken findings.

Key takeaway. An outbound feature becomes SSRF when input makes it cross an unauthorized boundary. Always distinguish the incoming and outgoing requests.

Check your understanding

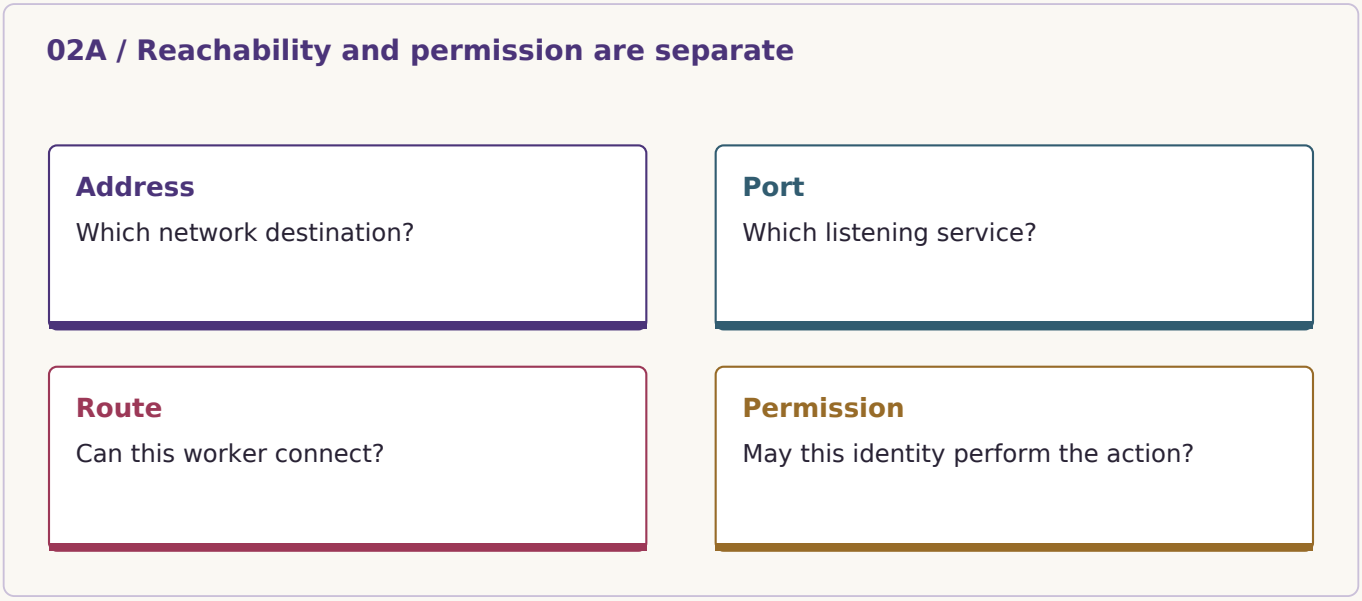
1.1 Does a working link preview prove SSRF?	**1.2** Whose network position matters for the second leg?	**1.3** Does the browser login cookie automatically travel onward?

Answer explanations are in the full ebook. Sources: CWE, OWASP, MDN

02. Addresses, ports, and routes

Learn to: Explain how a program reaches a service.

Think of it this way: An IP address is a building address. A port is a numbered service desk inside it.



A route permits contact; it does not grant application authorization. Both layers belong in the review.

A network is a set of reachable places

The internet joins many networks. A private application network is another set of connected systems. **Reachability** means a connection can get to a destination under the current routing and firewall rules. It is different from permission to use the destination's data.

An **IP address** identifies a network destination or interface in a routing context. IPv4 uses familiar dotted numbers. IPv6 uses a larger address space, usually written with hexadecimal groups. A **port** identifies a transport endpoint at that address. HTTPS often uses TCP port 443. A local administrative service might listen on a completely different port.

TCP provides an ordered byte stream. HTTP is one protocol that can travel over it. UDP is a different transport used by other services. HTTP/3 uses QUIC over UDP. Do not assume every network request is TCP merely because a teaching lab uses HTTP/1.1.

Local is local to the requester

The loopback address `127.0.0.1` points back to the current host's network stack. In many container arrangements, that means the container's network namespace rather than the whole physical machine. Some deployments deliberately share a host network. The same written address can therefore refer to different reachable services in different processes.

`0.0.0.0` is commonly used as a wildcard listen address. It is not a reliable universal spelling of a remote service. Reject unspecified destinations in a public fetcher. Do not teach readers to infer a successful connection from address text alone.

Read a connection as a tuple

For an ordinary TCP connection, record the source address and port plus the destination address and port. Also record the network namespace, proxy, and protocol. The source port is often chosen automatically. A destination allowlist should constrain the destination port explicitly when the product only needs one service.

A firewall can allow HTTPS outbound while blocking a private range. Another rule may allow all traffic to an internal subnet. Network address translation, or **NAT**, can change the source address seen by the destination. A callback from one public address does not uniquely identify the original worker.

Work through the example

Parcel Press runs a preview worker. The worker can reach a local health endpoint on `127.0.0.1:9000`, an internal billing service, and the internet. A customer can reach only the public web frontend. If the preview worker accepts arbitrary destinations, the customer may borrow connections that the frontend normally hides.

This does not mean the customer can read billing data. Billing may require an independent credential. It also does not mean the worker can run arbitrary programs. Reachability is one prerequisite. Authentication, protocol compatibility, method control, and response visibility are separate prerequisites.

Practice

List three destinations a worker needs and three it does not. Include ports. For each needed destination, ask whether the worker needs a credential as well as a route. A useful network policy is a statement of business need, not merely a list of addresses that happen to be reachable today.

Key takeaway. Record destination, port, route, and identity separately. Localhost belongs to the requester’s network context.

Check your understanding

2.1 Does access to an internal port prove data access?	**2.2** Why constrain destination ports?	**2.3** Why is a callback source address incomplete evidence?

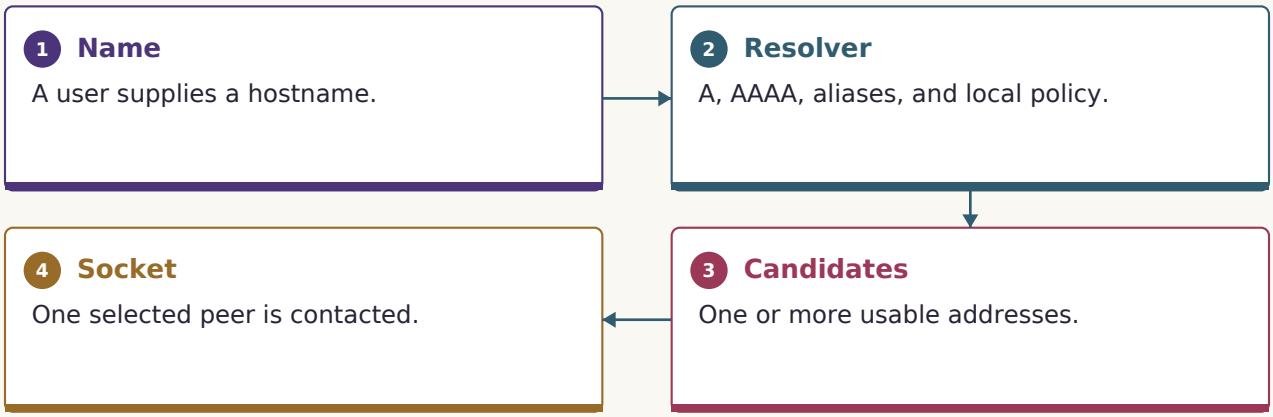
Answer explanations are in the full ebook. Sources: TCP, IP6, K8S

03. DNS is a lookup, not permission

Learn to: Understand names, records, aliases, and multiple answers.

Think of it this way: A directory gives a delivery address. It does not decide whether a delivery is allowed.

03A / A name is one step in the route



The security check must cover the address that the socket actually uses, including fallback candidates.

Names help people find machines

DNS, the Domain Name System, maps names to information. An A record contains an IPv4 address. An AAAA record contains an IPv6 address. A CNAME points to another name. An application often asks an operating-system resolver for addresses rather than sending DNS packets itself.

The operating system may consult a hosts file, a local cache, a corporate resolver, or another naming service. A direct DNS library and the system resolver can therefore produce different answers. Inspect the resolver used by the actual HTTP client, not just the resolver used by a validation helper.

One name can lead to several destinations

A hostname may have several addresses. A client can try another address after a failed connection, or race IPv4 and IPv6 connections. That improves availability. It also means validating one address while allowing the client to choose from a larger unchecked set is unsafe.

Names can change over time. DNS records include a time to live, or **TTL**, that influences caching. A short TTL does not guarantee when every cache refreshes. A long TTL does not make an attacker-owned hostname trustworthy. A security decision must be tied to the connection actually opened.

Internal and external answers

An organization may return a private address to internal workers and a public answer to outside users. This is often called split-horizon DNS. A tester’s laptop can therefore see an answer that says little about the worker’s destination.

In our fictional review, `media.vendor.example` resolves differently in the preview worker's environment. The reviewer must capture the worker's relevant answer and the connected peer. The hostname alone is not enough to prove an allowed public connection.

Observation	What it supports	What it does not establish
A DNS query reached your resolver	Some component attempted name resolution	An HTTP connection completed
An HTTP callback reached your server	Some component sent an HTTP request	It can reach every private service
A response body appeared in the preview	A readback path exists	The same path works for every destination

A first defense rule

When names are necessary, resolve under controlled conditions. Evaluate the usable answers for both families. Select an approved address and make the connection use that selected address. Do not validate a name and then let another layer silently resolve it again. Later chapters show why this is harder around proxies, retries, pools, and redirects.

DNSSEC can authenticate DNS data within its deployment model. It does not make an authenticated answer safe for your application. A legitimate owner can publish a private address. DNS over HTTPS similarly changes transport and privacy, not authorization.

Practice

Write down the difference between the name, the answer set, and the connected peer. Now imagine the answer changes after validation. Circle the exact step where your proposed defense still guarantees the permitted destination. If you cannot identify that step, the design needs a stronger connection boundary.

Key takeaway. DNS answers describe destinations. They do not authorize them. Follow the actual resolver and the actual peer.

Check your understanding

3.1 Does a DNS callback prove an HTTP request?	**3.2** What is wrong with checking only the first A record?	**3.3** Does DNSSEC prevent SSRF?

Answer explanations are in the full ebook. Sources: DNS, DNSNODE, HAPPY

Sources and further reading

Research cutoff: 26 October 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

CWE - MITRE CWE-918: Server-Side Request Forgery

Consulted. Definition and weakness framing. A classification is not proof that a particular application is exploitable.

https://cwe.mitre.org/data/definitions/918.html

OWASP - OWASP SSRF Prevention Cheat Sheet

Consulted. Destination validation, redirects, DNS and architecture guidance. Original examples and analysis in this book extend the lifecycle model; this is not a verbatim adaptation.

https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

MDN - MDN: Server-Side Request Forgery

Consulted. Browser-platform educational framing and distinction between legitimate fetching and a vulnerability.

https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/SSRF

TCP - RFC 9293: Transmission Control Protocol

Consulted for transport foundations. Read the relevant sections rather than treating a short analogy as a full TCP model.

https://www.rfc-editor.org/rfc/rfc9293.html

IP6 - RFC 4291: IPv6 Addressing Architecture

Targeted further reading. IPv6 address forms and types; combine with current special-purpose registries and deployment routing.

https://www.rfc-editor.org/rfc/rfc4291.html

DNS - RFC 1035: Domain Names - Implementation and Specification

Targeted further reading. DNS resource records and protocol foundations. System name resolution can use additional sources.

https://www.rfc-editor.org/rfc/rfc1035.html

DNSNODE - Node.js DNS API

Consulted current Node DNS documentation; this link pins the major line used by the lab. Review lookup versus protocol resolution and connector hooks.

https://nodejs.org/docs/latest-v24.x/api/dns.html

HAPPY - RFC 8305: Happy Eyeballs Version 2

Consulted. Multiple addresses, address families, and connection-selection context. The lab does not implement a complete Happy Eyeballs client.

https://www.rfc-editor.org/rfc/rfc8305.html

K8S - Kubernetes Network Policies

Consulted. Enforcement depends on the network implementation; host, loopback and established-session limitations matter.

https://kubernetes.io/docs/concepts/services-networking/network-policies/